

Rocket landing policy: system architecture

How the landing controller of the school TVC rocket is learned with reinforcement learning, what it is trained on, where each part lives in the code, and how it reaches the flight computer. All numbers are from the code and the trained run `runs/20260927_013945_default_smooth2`.

1. The system in one paragraph

A 1.5 kg model rocket flies up on one solid motor, falls back tail-first under a nose drag brake, and lands on legs using a second solid motor whose thrust direction is steered by a two-axis gimbal. A six-degree-of-freedom simulator with sensor models and a flight computer is wrapped as a reinforcement learning environment. PPO (Proximal Policy Optimization, an on-policy actor-critic method) trains a small fully connected network, the actor, to command the gimbal during the landing burn. A second network, the critic, exists only during training. Everything else on the flight computer stays rule-based: a PID flies the boost, a precomputed stopping-distance table decides when to light the landing motor, and a safety layer filters every command. The actor is exported as plain C arrays for the microcontroller.

Item	Value
Learning algorithm	PPO, Stable-Baselines3 2.4 on PyTorch 2.5 (CPU); not Q-learning, because the action is continuous
Networks	Actor and critic, separate MLPs (no graph or recurrent networks), 2 hidden layers of 32 tanh units each
What the policy controls	Gimbal angle in the landing burn phase only (configurable per phase in YAML)
Observation / action	13 estimated values per plane / 2 outputs per plane (gimbal, ignite vote)
Training data	Simulated flights only, generated on the fly: about 2,300 flights, 4 million samples
Result (50 paired flights)	Calm: 47/50 landed, same as the PID. 4 m/s gusty crosswind: 49/50 vs PID 47/50

2. Training architecture: where PPO and the networks are

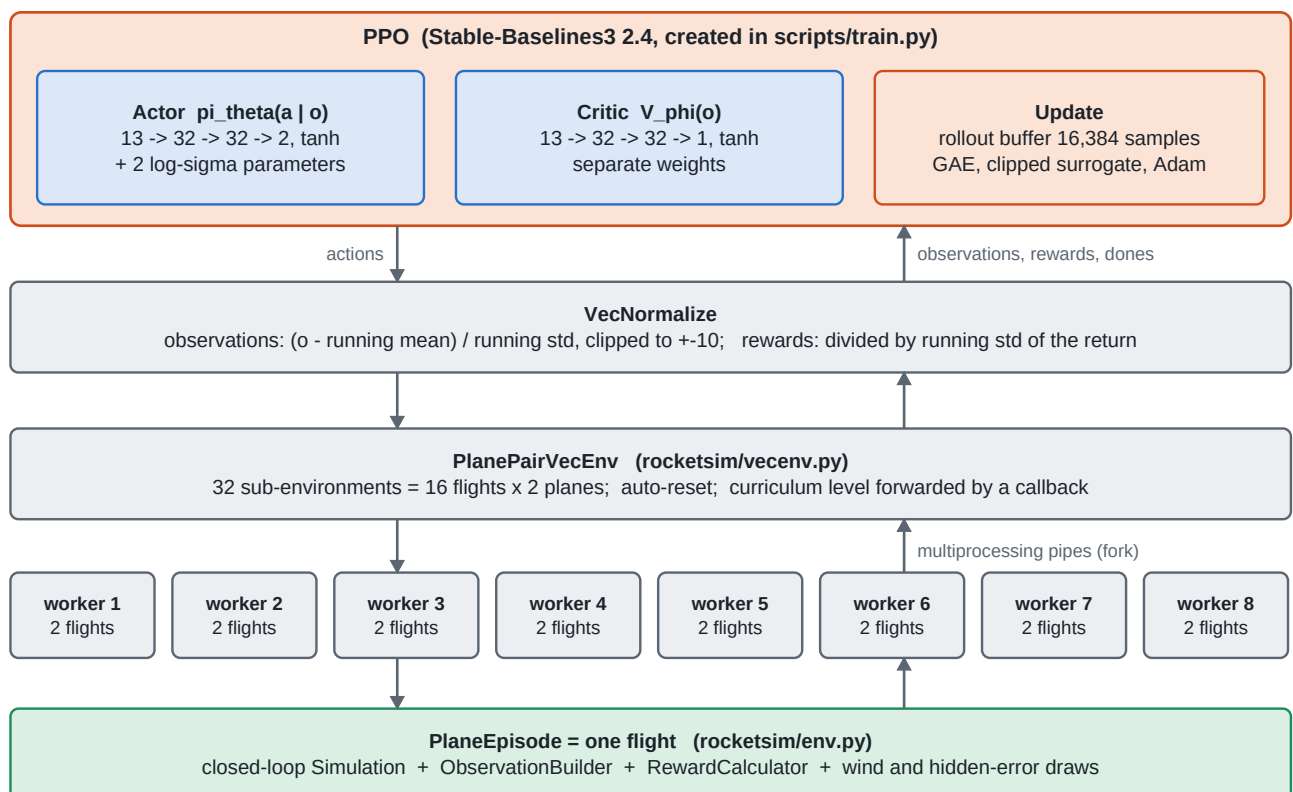


Figure 1. Data flow during training. PPO owns both networks. It sends 32 actions per step down and gets 32 observations and rewards back. Each flight contributes two samples per step, one per control plane (section 6).

- **PPO** is the Stable-Baselines3 PPO class, configured in `scripts/train.py` (`build_model`). We did not reimplement PPO.
- **The networks** are Stable-Baselines3's `ActorCriticPolicy` with `net_arch = {pi: [32, 32], vf: [32, 32]}`, so actor and critic have separate hidden layers. After training the actor is copied out into `rocketsim/policy.py` (`MlpPolicy`, plain numpy) and into C.

- **The environment** is our own code: `rocketsim/env.py` (one flight), `rocketsim/vecenv.py` (16 flights in 8 processes), around the closed-loop simulator `rocketsim/simulation.py`.

3. The two networks

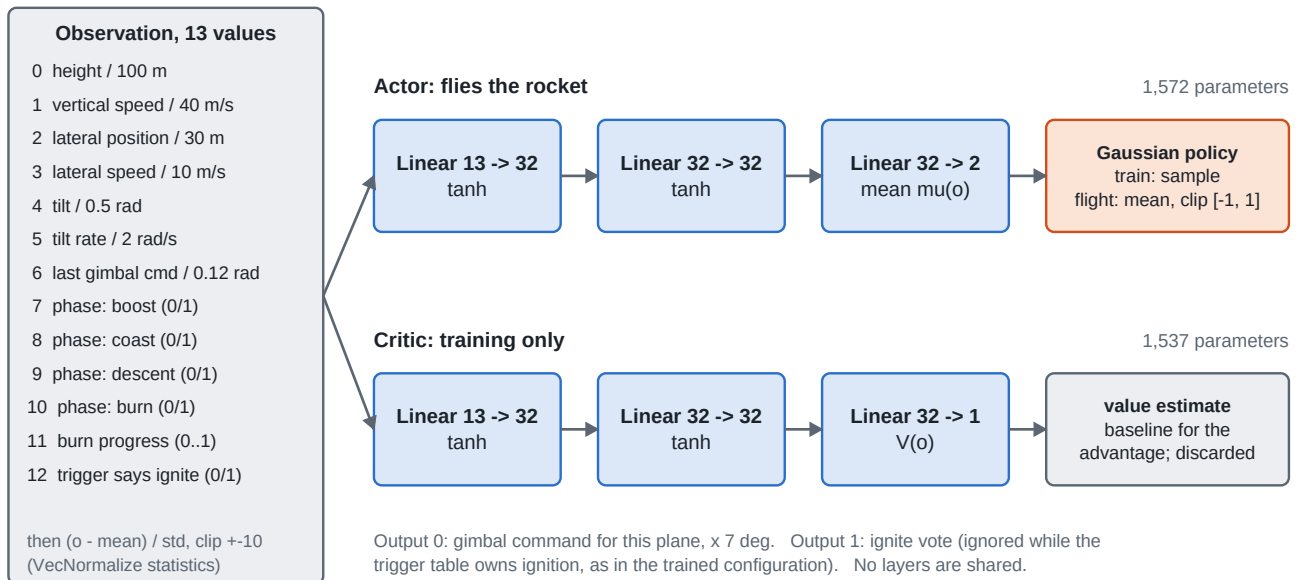


Figure 2. Actor and critic. Same input, separate weights. Only the actor goes to the rocket. The final learned standard deviations are 0.09 (gimbal) and 0.15 (ignite) in action units; in flight the mean is used, no sampling.

- The actor outputs the mean of a diagonal Gaussian; the log standard deviations are two free parameters, not a function of the input. They start at -1.5 (sigma 0.22, set by `log_std_init`) because the default sigma of 1 made the learned mean saturate and the gimbal flip between its limits.
- Actions live in $[-1, 1]$. The gimbal output is multiplied by the maximum gimbal angle, 7 degrees.
- The critic estimates the expected discounted return from the current observation. PPO uses it to compute advantages (how much better an action did than expected). It is never exported.

4. The environment: one step of the MDP

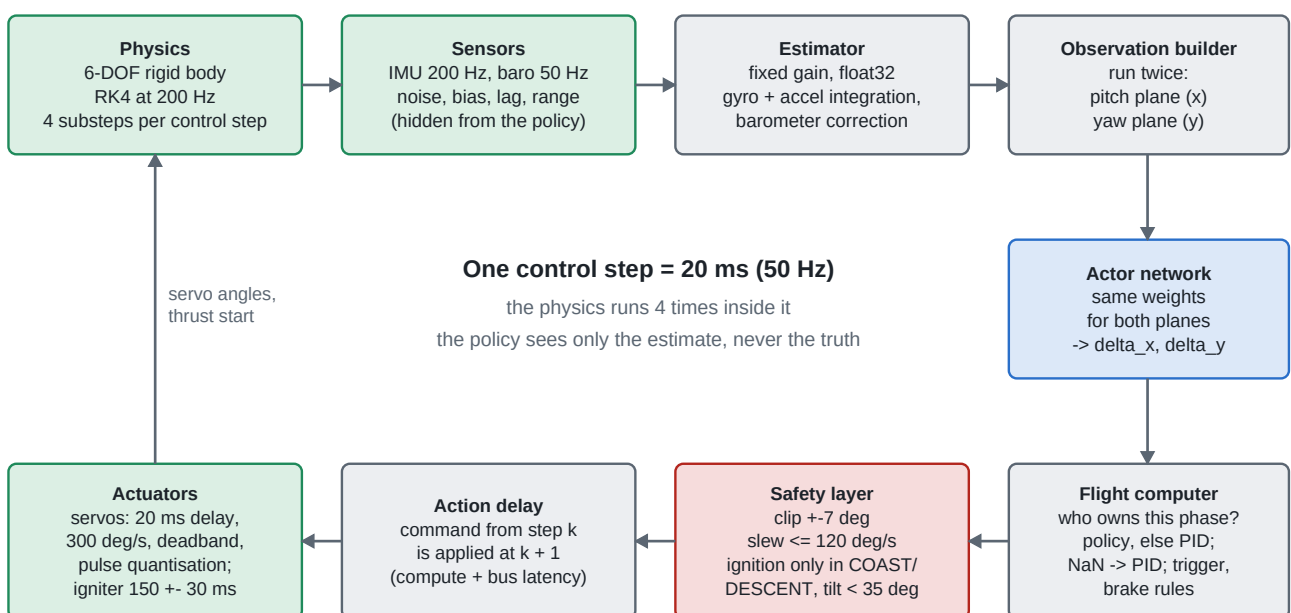


Figure 3. One control step. The loop runs at 50 Hz; the physics inside it at 200 Hz. The policy acts on the estimator's output with one step of delay, through the same safety layer and actuator models as the PID.

MDP element	In this system
State	True 6-DOF state (position, velocity, quaternion, body rates, burned propellant) plus sensor biases, filter state, servo and igniter state, wind. Not visible to the policy.
Observation	13 values from the estimator per plane (Figure 2), scaled by fixed constants, then normalised by running statistics. Partially observed; the MLP has no memory except the last gimbal command in the input.
Action	Per plane: gimbal command in $[-1, 1]$ and an ignite vote. Applied one control step later.
Transition	Deterministic physics given the seed; stochastic sensor noise, gusts, igniter delay and hidden errors.
Episode	One flight from the pad (calibration, launch) to touchdown, about 17 s = 850 control steps. Time limit 60 s.
Discount	$\gamma = 0.995$ per 20 ms step: effective horizon $1 / (1 - \gamma) = 200$ steps = 4 s, the length of the landing burn.

5. Who is in control during a flight

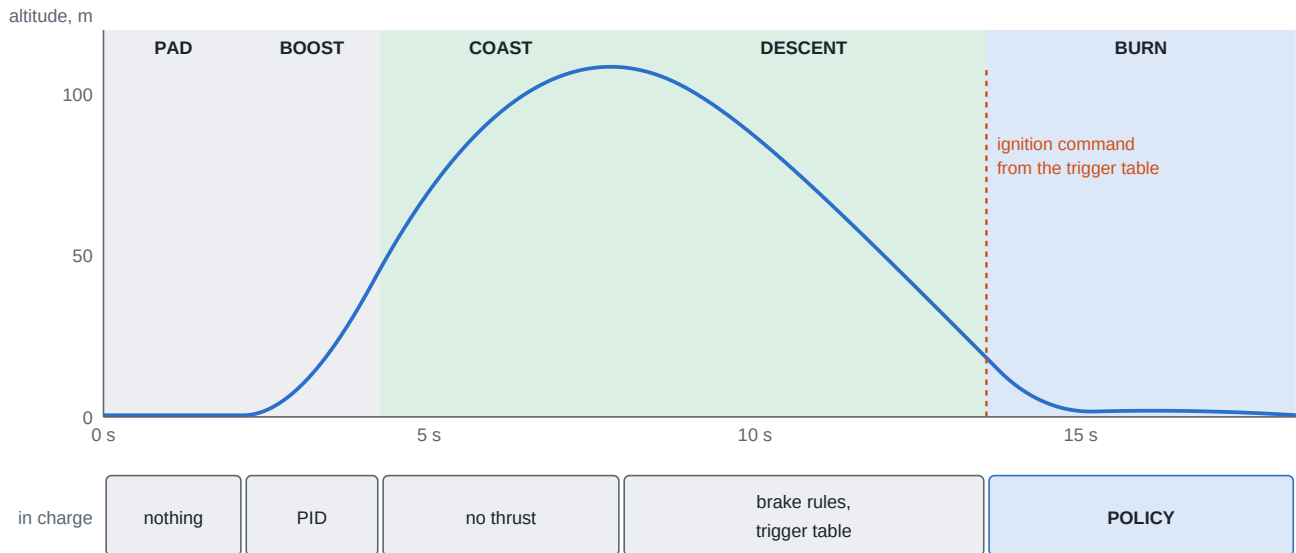


Figure 4. A real evaluation flight in calm air (altitude from the log). The policy steers only during the landing burn. The trigger table lights the motor; the PID flies the boost. Ownership is set per phase in the rocket YAML.

The rocket YAML has a controllers section: BOOST: pid | policy, LANDING_BURN: pid | policy, landing_ignition: trigger | policy. The trained run uses pid / policy / trigger. Outside the phases it owns, the policy's actions are computed but ignored, and only its phase earns the shaping rewards. A variant where the policy also flew the boost landed 42 of 50 in wind but learned to tumble the rocket in the coast, so it was rejected.

6. Two planes, one policy

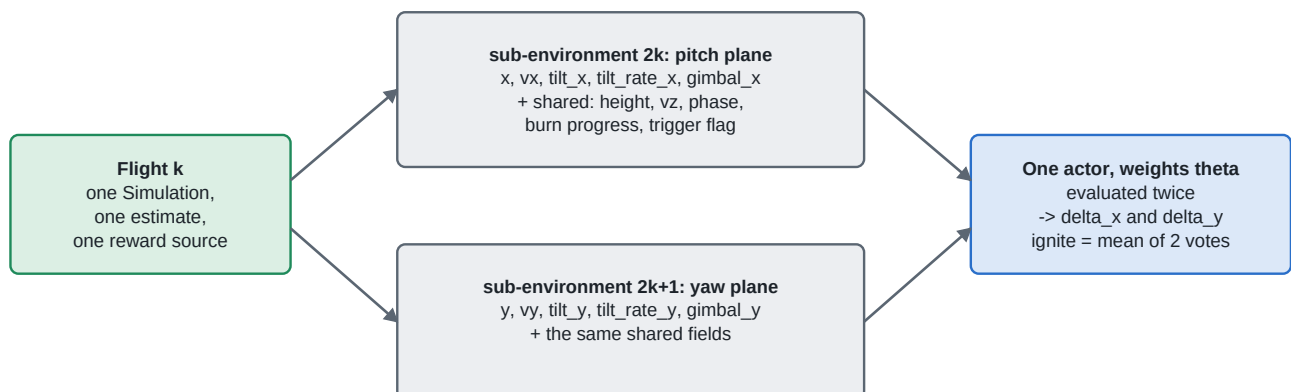


Figure 5. Parameter sharing. Each flight appears to PPO as two environments, one per control plane. One set of weights learns from both, as it will run on the board.

- The rocket is axisymmetric, so pitch and yaw obey the same equations. The policy is written for one plane and evaluated twice per step.
- This is a two-agent, shared-parameter decomposition: each plane gets its own reward (shared touchdown terms plus its own lateral terms). The planes are coupled through the physics, which the per-plane view treats as part of the environment.

- A roll angle rotates the two world-plane commands into the two servo commands before they go out.

7. What it trains on

No dataset. Every sample comes from a simulated flight generated during training. Each flight draws new random conditions (domain randomisation), so the policy cannot memorise one flight.

Randomised per flight	Range	Why
Landing motor thrust	+2 % of the curve	real motor batch spread
Ascent motor thrust	+5 %	changes apogee
Dry mass	+12 g	weighing error
Brake drag area	+5 %	petal drag uncertainty
Steady wind	0 to the stage maximum, any direction	crosswind
Gusts	Ornstein-Uhlenbeck, 2 s time constant	turbulence
Sensor bias, noise	from the sensor YAML (IMU, barometer)	real board quality
Igniter delay	150 +- 30 ms	real igniters vary

Curriculum stage (training progress)	Max steady wind	Gust std	Sensor noise	Hidden errors
0 to 20 %	0 m/s	0 m/s	x 0.2	off
20 to 50 %	3 m/s	1.0 m/s	x 1	half range
50 to 100 %	5 m/s	1.5 m/s	x 1	full range

Rewards, per plane (weights from configs/training/default.yaml)

Term	Weight	When
Landed inside the leg limits / crash / timeout	+40 / -40 / -40	once, at the end
Touchdown vertical speed	-3 per m/s	once
Touchdown sideways speed in this plane	-4 per m/s	once
Touchdown tilt	-0.4 per degree	once
Distance from the pad in this plane	-0.15 per m	once
Tilt / sideways speed / sideways position	-0.05 per rad / -0.01 per m/s / -0.002 per m	every step the policy steers
Gimbal change between steps	-8 per rad squared	every step the policy steers
Gimbal angle held	-0.1 per rad	every step the policy steers

8. PPO settings and what they mean

PPO maximises the clipped surrogate objective, with the advantage A from generalised advantage estimation (GAE):

$$L(\theta) = E[\min(r A, \text{clip}(r, 1 - \epsilon, 1 + \epsilon) A)], \quad r = \pi_{\theta}(a|o) / \pi_{\text{old}}(a|o), \quad \epsilon = 0.2$$

Setting	Value	Consequence
Parallel environments	16 flights x 2 planes = 32	8 worker processes
n_steps per environment	512	16,384 samples per update
batch size / epochs	2048 / 10	80 gradient steps per update
Total samples	4,000,000	about 244 updates, 2,300 flights
learning rate (Adam)	3e-4	constant
gamma / GAE lambda	0.995 / 0.95	horizon about 4 s
clip range / entropy coefficient	0.2 / 0	no entropy bonus
Initial log sigma	-1.5	sigma 0.22; ends at 0.09 and 0.15
Normalisation	VecNormalize, obs clip 10	statistics frozen into the export
Wall clock	34 minutes	about 1,950 samples per second on CPU

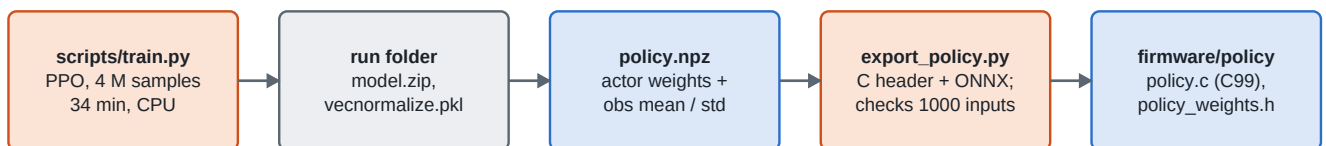
9. Results

The policy and the PID flew the same 50 seeds (100 to 149), with sensor noise and hidden errors on. The seeds are paired, so both controllers face identical conditions. Gimbal movement is the average change of the two gimbal commands per control step during the burn.

Conditions	Controller	Landed	Down speed	Side speed	Tilt	Miss	Gimbal move
Calm	Policy	47/50	1.53 m/s	0.30 m/s	0.2 deg	1.4 m	0.15 deg
Calm	PID	47/50	1.53 m/s	0.23 m/s	0.4 deg	1.3 m	0.07 deg
4 m/s wind, 1.5 m/s gusts	Policy	49/50	1.39 m/s	0.48 m/s	1.8 deg	19.5 m	0.17 deg
4 m/s wind, 1.5 m/s gusts	PID	47/50	1.36 m/s	0.60 m/s	1.3 deg	18.6 m	0.34 deg

Read with care: with 50 flights, 47 against 49 is within sampling noise; no significance test was run. The clearer differences are the lower sideways speed and half the gimbal movement in wind. Two earlier runs taught the main lessons: a policy trained without wind landed 3 of 50 windy flights, and a policy trained with the default exploration noise landed by steering bang-bang.

10. From training to the flight computer



Max difference C vs Python on 1000 random observations: 1.8e-7. ONNX: 1.8e-7. The rest of the flight computer in C (estimator, phases, safety, control loop) is the next stage and does not exist yet; today the policy runs in the Python simulator.

Figure 6. Deployment path. The critic and the optimiser stay behind; only the actor's weights and the observation normalisation are exported.

In the simulated flight computer a non-finite network output makes that plane fall back to the PID for that step. The C forward pass already refuses non-finite inputs; the fallback logic itself will be written with the rest of the firmware.

11. Where everything is in the code

What	File
PPO setup, training loop, curriculum callback, run folder	scripts/train.py
Evaluation against the PID on paired seeds	scripts/evaluate.py
Fly a trained policy in the simulator	scripts/fly_scripted.py --policy runs/<run>
Export to C and ONNX with the match check	scripts/export_policy.py
One flight as an RL episode (two plane views, rewards, wind draw)	rocketsim/env.py
32-environment vectorised wrapper, worker processes	rocketsim/vecenv.py
Observation fields and scales	rocketsim/observation.py
Reward terms	rocketsim/rewards.py
Curriculum levels	rocketsim/curriculum.py
Training YAML parsing (observation, rewards, ppo)	rocketsim/training_config.py
Actor as numpy, run-time controller for both planes	rocketsim/policy.py
Closed loop: sensors, flight computer, actuators, physics	rocketsim/simulation.py
Flight computer: phases, PID, trigger, ownership, fallback	rocketsim/flightcomputer.py
6-DOF dynamics, RK4	rocketsim/physics.py
Estimator (float32), sensors, safety	rocketsim/estimator.py, sensors.py, safety.py
Actor forward pass in C99	firmware/policy/policy.c
All settings	configs/rockets/example_tvc.yaml, configs/training/default.yaml

12. Simplifications a reviewer should know

- Only simulated data. The simulator has not yet been checked against a real flight; every rocket parameter is still a placeholder.
- Partial observability is handled only by the estimator and the last-command input; there is no recurrent network.
- Steps outside the policy's phases enter the rollout buffer even though the action has no effect. They carry no shaping reward but add variance.
- Timeouts are treated as terminal, not truncated. In evaluation none occurred.
- The two-plane decomposition ignores coupling between planes inside the policy; it relies on the rocket being axisymmetric.
- One rocket configuration, one seed for training. Run-to-run variance across training seeds has not been measured.

13. Reproduce

```
python scripts/train.py --training configs/training/default.yaml
python scripts/evaluate.py runs/<run> --episodes 50 --sim configs/training/windy.yaml
python scripts/export_policy.py runs/<run>
```